

Informe del módulo de base de datos vectorial

CSDI RAG Engine

1 Módulo de base de datos vectorial

El módulo de base de datos vectorial es el componente responsable de transformar fragmentos de texto en representaciones numéricas densas, almacenarlas de forma persistente y permitir recuperarlas por similitud semántica. Su función dentro del sistema RAG es responder a la pregunta: dado el vector de una consulta, ¿qué fragmentos del corpus son semánticamente más cercanos a ella?

Para cumplir esa función, el módulo integra tres tecnologías complementarias. Un modelo de embeddings transforma texto en vectores de alta dimensión que codifican significado semántico. Una estructura de índice especializada (*HNSW*) permite buscar entre esos vectores de forma eficiente sin comparar contra todos los documentos. Una base de datos relacional persiste los vectores entre reinicios y actúa como fuente de verdad para reconstruir el índice en memoria cuando es necesario.

La separación entre estas tres responsabilidades es deliberada: el modelo de embeddings puede cambiar sin afectar la estructura de búsqueda, y la base de datos puede reconstruir el índice en memoria si el archivo de índice en disco se corrompe o no existe.

Componentes del módulo. El módulo se organiza en cinco componentes con responsabilidades claramente delimitadas:

- **EmbeddingModel:** genera los vectores a partir de texto usando el modelo BAAI/bge-m3.
- **FaissIndex:** encapsula el índice HNSW de FAISS y sus operaciones de adición y búsqueda.
- **VectorStore:** mantiene en memoria el mapeo bidireccional entre identificadores de posición FAISS (*vector_id*) e identificadores de documento (*doc_id*).
- **VectorIndexBuilder:** coordina la indexación de nuevos documentos: codificación por lotes, actualización del índice y persistencia.
- **VectorRetriever:** ejecuta búsquedas sobre el índice y filtra documentos eliminados antes de devolver resultados.

2 Modelo de embeddings

Modelo seleccionado: BAAI/bge-m3. El modelo de embeddings utilizado es BAAI/bge-m3, cargado mediante la biblioteca `sentence_transformers`. Produce vectores de 1024 dimensiones con tipo `float32`. La elección de este modelo responde al perfil multilingüe del corpus: fue entrenado sobre datos en más de cien idiomas y produce representaciones compartidas entre lenguas, de modo que una consulta formulada en español y un fragmento escrito en inglés sobre el mismo concepto quedan proyectados a regiones cercanas del espacio vectorial. Un modelo monolingüe no puede satisfacer este requisito por diseño.

Carga diferida y singleton. El modelo no se carga al arrancar el servidor. La clase `EmbeddingModel` implementa un patrón de carga diferida (*lazy loading*) protegido por un `threading.Lock`: el modelo se descarga y carga en memoria la primera vez que se invoca `encode()`, y la instancia resultante se reutiliza en todas las llamadas posteriores. Este diseño tiene dos ventajas concretas: el servidor arranca más rápido porque no espera a que el modelo (~570 MB para `bge-m3`) se cargue en memoria, y evita que múltiples hilos inicialicen el modelo simultáneamente durante el primer acceso.

El modelo de embeddings es compartido entre el índice del corpus principal y el índice del caché web. Esto garantiza que los vectores de ambos espacios sean comparables y que no se duplique el consumo de memoria.

Normalización L2 y similitud coseno. Todos los vectores generados por el modelo se normalizan con norma L2 antes de insertarse en el índice. Esta normalización no es opcional: es la que hace que el producto interno entre dos vectores normalizados sea equivalente a la similitud coseno entre ellos. FAISS usa la métrica de producto interno (`METRIC_INNER_PRODUCT`), que sobre vectores normalizados produce exactamente los mismos rankings que la similitud coseno, pero es más rápida de calcular porque evita la raíz cuadrada de la norma.

Codificación asimétrica con prefijo. El modelo soporta codificación asimétrica mediante un prefijo de consulta configurable (`QUERY_PREFIX`). Cuando se codifica una consulta, el prefijo se antepone al texto antes de la codificación; cuando se codifican documentos para indexación, el prefijo no se aplica. Esta asimetría permite ajustar el espacio vectorial para que las consultas queden posicionadas de forma más cercana a los documentos que las responden. El valor por defecto es cadena vacía (sin asimetría), lo que produce un espacio simétrico donde consultas y documentos se codifican de la misma forma.

3 Índice FAISS HNSW

Estructura del índice. El índice vectorial es un `faiss.IndexHNSWFlat` con métrica de producto interno. HNSW (*Hierarchical Navigable Small World*) es una estructura de grafo jerárquico que organiza los vectores en capas: las capas superiores tienen pocos nodos conectados por aristas largas (navegación rápida) y la capa inferior contiene todos los nodos con sus vecinos más cercanos reales. Una búsqueda navega desde la capa superior hacia la inferior, reduciendo progresivamente el conjunto de candidatos explorados.

Parámetros de construcción. El índice se construye con tres parámetros configurables:

- **M=32**: número máximo de conexiones por nodo en el grafo. Un valor mayor produce un grafo más denso con mayor tasa de recuperación pero más memoria y tiempo de construcción. El valor 32 es la configuración de alta calidad recomendada por Malkov & Yashunin (2020) para corpus de tamaño medio donde la precisión en los primeros resultados es prioritaria.
- **ef_construction=200**: tamaño de la lista dinámica de candidatos durante la construcción del grafo. Un valor más alto produce un grafo mejor conectado a costa de mayor tiempo de indexación. El valor 200 es el doble del valor por defecto de FAISS (100) y garantiza que los vecinos reales de cada punto sean explorados con alta probabilidad durante la construcción.
- **ef_search=50**: tamaño de la lista dinámica de candidatos durante la búsqueda. Controla el equilibrio entre exhaustividad y velocidad en tiempo de consulta. Un valor de 50 recupera la mayoría de los vecinos reales para corpus de tamaño medio sin imponer una latencia apreciable.

Adición de vectores y restricciones del índice. Los vectores se añaden al índice en el orden en que son procesados por el builder. El índice HNSW de FAISS asigna identificadores enteros secuenciales (*vector_ids*) a cada vector insertado: el primero recibe el id 0, el segundo el 1, y así sucesivamente. Este índice no soporta eliminación de vectores ni actualización de posiciones una vez insertados. Las eliminaciones se gestionan mediante una estrategia de *soft delete* descrita en la sección de eliminaciones.

Persistencia del índice en disco. Si la variable `FAISS_INDEX_PATH` está configurada, el índice se serializa a disco mediante `faiss.write_index()` después de cada lote de documentos procesado. La persistencia en disco activa una ruta de carga rápida al arrancar: si el archivo existe, el índice se carga directamente sin reconstruir desde la base de datos. Esta ruta evita el costo de recodificar todos los embeddings almacenados al reiniciar el servidor.

4 VectorStore: mapeo en memoria

El `VectorStore` es la estructura de datos en memoria que resuelve el problema de identidad entre FAISS y el sistema de documentos. FAISS identifica vectores por su posición entera en el índice (*vector_id*); el sistema identifica documentos por su *chunk_id* textual. El `VectorStore` mantiene dos diccionarios: uno de *vector_id* a *doc_id* y otro en sentido inverso. También mantiene el conjunto de *doc_ids* marcados como eliminados.

La integridad de estos mapeos es crítica: si la correspondencia entre *vector_id* y *doc_id* es incorrecta, una búsqueda puede devolver un fragmento completamente distinto al que corresponde al vector encontrado. Por eso el módulo valida al arrancar que el número de vectores en el índice FAISS cargado desde disco coincida exactamente con el número de documentos en la base de datos antes de confiar en el índice de disco.

5 Flujo de indexación

Encolamiento y procesamiento por lotes. Cuando el sistema recibe un nuevo fragmento para indexar, el `VectorIndexBuilder` no lo codifica inmediatamente. El texto y su identificador se acumulan en un buffer en memoria. Cuando el buffer alcanza el tamaño de lote (`VECTOR_BATCH_SIZE=64`), o cuando se solicita un vaciado explícito, el builder codifica todos los textos del buffer en una sola llamada al modelo. Esta estrategia es significativamente más eficiente que codificar un texto a la vez: los modelos de embeddings están optimizados para procesar matrices de vectores, y el overhead de inicialización de cada llamada se amortiza entre todos los textos del lote.

Secuencia de operaciones en un flush. El vaciado del buffer ejecuta los pasos siguientes en orden:

1. **Codificación:** los textos del buffer se pasan al modelo de embeddings, que devuelve una matriz `numpy` de forma $(n, 1024)$ con vectores normalizados en `float32`.
2. **Indexación FAISS:** la matriz se añade al índice HNSW con `faiss.IndexHNSWFlat.add()`, que asigna *vector_ids* secuenciales.
3. **Actualización del VectorStore:** los pares (*vector_id*, *doc_id*) se registran en los diccionarios bidireccionales del store.

4. **Persistencia en base de datos:** los pares (`doc_id`, `vector`) se insertan en la tabla `vector_documents` mediante un `INSERT` masivo.
5. **Actualización de metadatos:** se actualiza `vector_index_metadata` con el nuevo conteo de vectores y el modelo utilizado.
6. **Escritura a disco:** si `FAISS_INDEX_PATH` está configurado, el índice completo se serializa a disco.

Todo este proceso está protegido por un `threading.RLock` que impide que búsquedas concurrentes accedan al índice mientras está siendo modificado.

6 Inicialización al arrancar

Al arrancar el servidor, el `VectorIndexBuilder` ejecuta su método `start()`, que reconstruye el estado completo del índice siguiendo una de tres rutas en orden de preferencia:

Ruta 1: carga desde disco. Si `FAISS_INDEX_PATH` existe como archivo válido, el índice se deserializa con `faiss.read_index()`. Antes de usarlo, se validan dos invariantes: que la dimensión del índice cargado coincida con `VECTOR_DIMENSION=1024`, y que el número de vectores en el índice coincida con el número de documentos activos en la tabla `vector_documents`. Si ambas validaciones pasan, el índice de disco es válido y se carga solo el listado de `doc_ids` desde la base de datos para reconstruir el `VectorStore`. Esta ruta es la más rápida: evita recodificar embeddings y carga solo metadatos ligeros.

Ruta 2: reconstrucción desde base de datos. Si el archivo de disco no existe, o si alguna validación de la ruta 1 falla, el builder lee todos los embeddings almacenados desde la tabla `vector_documents` en lotes de 1000 registros y los inserta en un índice FAISS recién creado. Esta ruta es más lenta porque requiere leer todos los vectores de la base de datos, pero garantiza que el índice en memoria sea consistente con la fuente de verdad persistente.

Ruta 3: índice vacío. Si no hay documentos en la base de datos, el builder inicializa un índice FAISS vacío listo para recibir los primeros documentos.

Validación de metadatos del modelo. Antes de ejecutar cualquiera de las tres rutas, el builder verifica que los metadatos almacenados en `vector_index_metadata` sean compatibles con la configuración actual. Si el modelo o la dimensión registrados en los metadatos difieren de los valores actuales, el sistema lanza un `RuntimeError` con un mensaje explícito que indica que las tablas de vectores deben ser truncadas antes de reiniciar. Este control evita que vectores generados con un modelo diferente se mezclen con los del modelo actual, lo que produciría rankings de similitud incorrectos.

7 Flujo de búsqueda

Una consulta de búsqueda vectorial sigue estos pasos:

1. **Codificación de la consulta:** el texto de la consulta se normaliza y codifica con `EmbeddingModel.encode_query()` que antepone el prefijo configurado antes de llamar al modelo. El resultado es un vector de forma $(1, 1024)$ en `float32`.

2. **Búsqueda en FAISS:** bajo el `RLock`, se llama a `FaissIndex.search(query_vector, top_k)`, que devuelve dos matrices: una de puntuaciones de producto interno (*scores*) y una de *vector_ids*, ambas de forma (1, top_k).
3. **Resolución de identificadores:** para cada *vector_id* devuelto, se consulta el `VectorStore` para obtener el *doc_id* correspondiente. Los *vector_ids* negativos (padding de FAISS cuando hay menos resultados que top_k) se ignoran.
4. **Filtrado de eliminaciones:** los *doc_ids* que están en el conjunto de documentos eliminados se excluyen del resultado sin devolver error.
5. **Construcción del resultado:** los pares (*doc_id*, *score*) válidos se empaquetan como `VectorResult` y se devuelven al `HybridRetriever`.

8 Persistencia en base de datos

Esquema de tablas del corpus principal. La tabla `vector_documents` almacena un registro por fragmento indexado. Sus columnas son: un identificador entero autoincremental (*id*), el identificador textual del fragmento (*doc_id*, único e indexado), el embedding como tipo `pgvector(1024)`, la marca temporal de indexación y un campo `deleted_at` nullable que implementa el soft delete.

La tabla `vector_index_metadata` almacena un único registro (*id*=1) con los parámetros del índice actual: nombre del modelo de embeddings, dimensión de vectores, tipo de índice FAISS, parámetros HNSW y conteo total de vectores. Este registro se actualiza en cada flush y sirve para validar la compatibilidad del índice al arrancar.

Uso de pgvector. Los embeddings se almacenan usando la extensión `pgvector` de PostgreSQL, que define el tipo `Vector(1024)`. Esta extensión permite almacenar vectores de punto flotante de forma compacta y expone operadores SQL para búsquedas de similitud (<-> para distancia L2, <#> para producto interno negativo). En el diseño actual, las búsquedas de similitud se realizan exclusivamente a través de FAISS en memoria, no mediante consultas SQL con pgvector. La base de datos actúa como almacén persistente de los vectores, no como motor de búsqueda.

9 Eliminación de documentos

Estrategia de soft delete. FAISS no soporta la eliminación de vectores individuales una vez insertados en un índice HNSW. El módulo resuelve esta limitación con una estrategia de *soft delete*: los vectores permanecen en el índice FAISS, pero cuando un documento es eliminado, su *doc_id* se añade al conjunto `deleted_doc_ids` del `VectorStore` y se establece `deleted_at` en la base de datos. Las búsquedas filtran los resultados de FAISS contra este conjunto antes de devolverlos, de modo que los documentos eliminados nunca aparecen en los resultados aunque sus vectores aún estén en el índice.

Reclamación de espacio. Los slots ocupados por documentos eliminados no se recuperan en tiempo de ejecución. La reclamación ocurre en el próximo reinicio del servidor: como la ruta de reconstrucción desde base de datos solo carga documentos con `deleted_at IS NULL`, el índice reconstruido no contiene los vectores eliminados y el espacio queda libre. Si el servidor usa la ruta de carga desde disco y el número de documentos activos coincide con el índice de disco, los documentos eliminados previamente tampoco aparecerán porque el filtro del `VectorStore` sigue activo.

10 Índice vectorial del caché web

El caché web mantiene un índice vectorial paralelo al del corpus principal, con las mismas características técnicas pero en tablas separadas: `web_cache_vector_documents` y `web_cache_vector_index_metadata`. El `VectorIndexBuilder` del caché web es una instancia independiente que opera sobre un `WebCacheVectorRepository` distinto, garantizando que los vectores web no se mezclen con los del corpus curado.

La diferencia más relevante respecto al índice principal es que la tabla `web_cache_vector_documents` no tiene columna `deleted_at`: el caché web no implementa soft delete porque sus documentos se gestionan por reemplazo, no por eliminación selectiva. El modelo de embeddings, en cambio, es la misma instancia compartida con el índice principal: no se carga un segundo modelo en memoria, lo que mantiene el consumo de recursos bajo.

El índice del caché web no persiste en disco (`FAISS_INDEX_PATH` no se aplica al caché). En cada reinicio se reconstruye desde la base de datos, lo que es aceptable porque el número de documentos en el caché crece más lentamente que el corpus principal.

11 Configuración y parámetros clave

Variable	Default	Descripción
<code>EMBEDDING_MODEL</code>	<code>BAAI/bge-m3</code>	Modelo de sentence-transformers
<code>VECTOR_DIMENSION</code>	1024	Dimensión de los vectores
<code>FAISS_INDEX_TYPE</code>	<code>HNSW</code>	Tipo de índice FAISS
<code>HNSW_M</code>	32	Conexiones máximas por nodo
<code>HNSW_EF_CONSTRUCTION</code>	200	Candidatos durante construcción
<code>HNSW_EF_SEARCH</code>	50	Candidatos durante búsqueda
<code>VECTOR_BATCH_SIZE</code>	64	Tamaño del lote de codificación
<code>QUERY_PREFIX</code>	(vacío)	Prefijo para codificación de consultas
<code>FAISS_INDEX_PATH</code>	(vacío)	Ruta de persistencia del índice FAISS

La variable `VECTOR_DIMENSION` debe coincidir con la dimensión de salida del modelo configurado en `EMBEDDING_MODEL`. Si se cambia el modelo por uno de dimensión distinta sin actualizar `VECTOR_DIMENSION`, el sistema lanzará un error al intentar insertar vectores de dimensión incorrecta en el índice. Si se cambia el modelo manteniendo `VECTOR_DIMENSION`, el sistema arrancará pero los vectores del índice existente y los nuevos serán incomparables, produciendo rankings de similitud sin sentido. El mecanismo de validación de metadatos al arrancar detecta este segundo caso y bloquea el inicio.

12 Justificación técnica de la solución

El módulo de base de datos vectorial existe porque la recuperación léxica (BM25) no puede por sí sola responder consultas que usan vocabulario distinto al del corpus. Un sistema RAG que solo usa BM25 falla ante paráfrasis, consultas en idioma diferente al de los documentos o preguntas conceptuales. Los embeddings densos resuelven este problema proyectando texto con significado equivalente a regiones cercanas del espacio vectorial independientemente del vocabulario exacto.

Por qué HNSW sobre un índice plano. La alternativa más simple a HNSW es un índice plano: se compara el vector de consulta contra todos los vectores del corpus en tiempo lineal. Para corpus pequeños esto es exacto y suficiente, pero escala mal: con 100.000 fragmentos y vectores de

1024 dimensiones, cada búsqueda requeriría calcular 100 millones de productos internos. HNSW reduce ese costo a un subconjunto logarítmico del corpus sin pérdida apreciable de calidad para los parámetros configurados. Los índices de cuantización como IVF (Inverted File Index) también son más rápidos que el índice plano, pero requieren una fase de entrenamiento sobre el corpus antes de poder usarse, lo que añade complejidad al proceso de indexación y puede perder calidad en corpus pequeños o poco densos.

Por qué la base de datos como fuente de verdad. El índice FAISS en memoria es una estructura efímera: no sobrevive reinicios del servidor a menos que se persista explícitamente a disco. La base de datos relacional actúa como la fuente de verdad persistente desde la cual el índice puede reconstruirse en cualquier momento. Esta separación de responsabilidades garantiza que los datos no se pierdan ante un fallo del servidor, y que el sistema pueda recuperarse automáticamente sin intervención manual si el archivo de índice en disco se corrompe o se elimina. El costo de la reconstrucción desde base de datos es asumible en el arranque porque es una operación infrecuente; el costo de una búsqueda posterior, que opera sobre el índice en memoria, no se ve afectado.

Por qué soft delete y no eliminación directa. La limitación de FAISS que impide eliminar vectores individuales en HNSW podría sortearse reconstruyendo el índice completo cada vez que se elimina un documento, pero eso haría de cada eliminación una operación $O(n)$ sobre el corpus. El soft delete convierte cada eliminación en una operación $O(1)$ (añadir un identificador a un conjunto en memoria y actualizar un campo en base de datos) a costa de que los vectores eliminados ocupen espacio en el índice hasta el próximo reinicio. Para el dominio del sistema, donde las eliminaciones son infrecuentes y el corpus no crece sin límite, este compromiso es adecuado.

13 Limitaciones del módulo

El modelo de embeddings se carga en CPU sin posibilidad de configurar una GPU mediante variables de entorno. Para corpus con decenas de miles de documentos, la codificación en CPU puede ser el cuello de botella del proceso de indexación masiva inicial. La búsqueda sobre el índice HNSW en memoria no se ve afectada por este límite porque FAISS opera eficientemente en CPU.

El índice HNSW no soporta actualizaciones incrementales de vectores. Si el texto de un fragmento cambia, el vector antiguo permanece en el índice y el nuevo se añade como un registro adicional, con el anterior siendo soft-deleted. Esto puede incrementar el tamaño del índice con el tiempo en corpus con alta tasa de actualización.

La validación de compatibilidad del modelo al arrancar es una comprobación por nombre y dimensión, no una comprobación por hash del modelo. Si se instala una versión diferente del mismo modelo con el mismo nombre pero pesos distintos, el sistema no lo detectará y los vectores del índice existente y los nuevos serán incomparables.

El caché web no tiene política de expiración de vectores. Fragmentos de búsquedas antiguas permanecen en `web_cache_vector_documents` indefinidamente, lo que puede hacer crecer el índice del caché sin límite y degradar el rendimiento de búsqueda sobre él con el tiempo.